

Instituto Nacional de Astrofísica, Óptica y Electrónica

Coordinación de Ciencias Computacionales

Laboratorio de Visión por Computadora

Reporte de Estancia de Investigación

Proyecto:

Normalización y alineación automática de la forma de la región pulmonar integrada con selección de características discriminantes para detección automática de neumonía y COVID-19

Estudiante: Rafael Alejandro Cruz Ovando

Director: Dr. Leopoldo Altamirano Robles

Institución de origen: Benemérita Universidad Autónoma de Puebla

Período: 9 de octubre – 9 de noviembre de 2025

Enero 2026

Índice

Datos Generales	3
1. Resumen Ejecutivo	3
1.1. Resultados Principales	3
1.2. Contribuciones	3
2. Introducción	4
2.1. Contexto del Proyecto	4
2.2. Importancia de los Puntos de Referencia Pulmonares	4
2.3. Estructura de 15 Puntos de Referencia	4
3. Metodología Implementada	4
3.1. Preparación de Datos	4
3.1.1. Dataset Utilizado	4
3.1.2. Preprocesamiento	5
3.1.3. Aumentación de Datos	5
3.2. Arquitectura del Modelo	5
3.2.1. Backbone: ResNet-18	5
3.2.2. Coordinate Attention	6
3.3. Funciones de Pérdida	6
3.3.1. Wing Loss (Pérdida Principal)	6
3.3.2. Central Alignment Loss	7
3.3.3. Soft Symmetry Loss	7
3.3.4. Combined Loss	8
3.4. Plan de Entrenamiento	8
3.4.1. Fase 1: Cabeza Congelada	8
3.4.2. Fase 2: Fine-tuning Diferenciado	8
3.5. Estrategia de Ensemble	9
3.5.1. Test-Time Augmentation (TTA)	9
3.6. Normalización Geométrica (Warping)	10
3.6.1. Generalized Procrustes Analysis (GPA)	10
3.6.2. Triangulación de Delaunay	10
3.6.3. Warping Afín por Partes	11
3.6.4. Preprocesamiento SAHS para Clasificación	11
4. Resultados y Análisis	12
4.1. Métricas de Detección de Puntos de Referencia	12
4.1.1. Error Global	12
4.1.2. Error por Categoría	12
4.1.3. Error por Punto de Referencia	13
4.2. Clasificación	13
4.2.1. Validación Cruzada (5-fold)	13
4.3. Objetivos Cumplidos	14
4.3.1. Extensiones Realizadas (Más allá del Plan)	14

5. Entregables Producidos	14
5.1. Código Fuente	14
5.2. Modelos Entrenados	15
5.3. Configuraciones Reproducibles	15
6. Cronograma Ejecutado	15
6.1. Mapeo Actividades vs Plan Original	15
6.2. Sesiones de Desarrollo	15
7. Conclusiones	16
7.1. Objetivos Cumplidos	16
7.2. Contribuciones Técnicas	16
7.3. Limitaciones Identificadas	16
7.4. Trabajo Futuro	16

Datos Generales

Campo	Información
Proyecto	Normalización y alineación automática de la forma de la región pulmonar integrada con selección de características discriminantes para detección automática de neumonía y COVID-19
Director	Dr. Leopoldo Altamirano Robles
Estudiante	Rafael Alejandro Cruz Ovando
Período	9 de octubre – 9 de noviembre de 2025
Lugar	Laboratorio de Visión por Computadora, INAOE
Institución de origen	Benemérita Universidad Autónoma de Puebla (BUAP)

1. Resumen Ejecutivo

Este reporte documenta el trabajo realizado durante la estancia de investigación en el Laboratorio de Visión por Computadora del INAOE. Se desarrolló un sistema completo para la detección automática de puntos de referencia pulmonares y normalización geométrica de radiografías de tórax para clasificación de COVID-19.

1.1 Resultados Principales

Métrica	Valor Obtenido
Error de puntos de referencia (ensemble)	3.61 px
Error de puntos de referencia (mejor individual)	4.04 px
Accuracy clasificación	98.60 % (CV)
F1-macro validación cruzada	98.00 % $\pm$ 0.36 %

Cuadro 1: Métricas principales del sistema desarrollado.

1.2 Contribuciones

- Sistema de detección de puntos de referencia:** Ensemble de 4 modelos ResNet-18 con Coordinate Attention y Test-Time Augmentation
- Normalización geométrica:** Proceso de warping afín por partes usando triangulación de Delaunay
- Funciones de pérdida geométricas:** Wing Loss con restricciones de simetría y alineación central
- Infraestructura reproducible:** Sistema de configuraciones JSON y documentación exhaustiva

2. Introducción

2.1 Contexto del Proyecto

La pandemia de COVID-19 resaltó la necesidad de herramientas de diagnóstico rápido y automatizado. Las radiografías de tórax ofrecen una alternativa accesible a las pruebas PCR, pero su interpretación requiere experiencia especializada.

Este proyecto aborda dos desafíos fundamentales:

1. **Detección de puntos de referencia anatómicos:** Localización automática de 15 puntos que definen el contorno pulmonar
2. **Normalización geométrica:** Transformación de imágenes a una forma estándar que elimina variabilidad anatómica

2.2 Importancia de los Puntos de Referencia Pulmonares

La región pulmonar en radiografías presenta alta variabilidad inter-paciente debido a:

- Diferencias anatómicas individuales
- Posicionamiento durante la adquisición
- Rotaciones y escalas variables

La detección precisa de puntos de referencia permite normalizar estas variaciones, mejorando la robustez de sistemas de clasificación downstream.

2.3 Estructura de 15 Puntos de Referencia

El sistema utiliza 15 puntos de referencia que definen el contorno pulmonar bilateral:

Eje central vertical:	L1 (apex) -> L9 -> L10 -> L11 -> L2 (base)
Contorno izquierdo:	L12, L3, L5, L7, L14
Contorno derecho:	L13, L4, L6, L8, L15
Pares simetricos:	(L3,L4), (L5,L6), (L7,L8), (L12,L13), (L14,L15)

3. Metodología Implementada

3.1 Preparación de Datos

3.1.1 Dataset Utilizado

Característica	Valor
Dataset base	COVID-19 Radiography Dataset
Total imágenes	15,153
Clases	COVID, Normal, Viral_Pneumonia
Anotaciones	15 puntos de referencia por imagen
División	75 % train / 15 % val / 10 % test

Cuadro 2: Características del dataset utilizado.

3.1.2 Preprocesamiento

El proceso de preprocesamiento para **detección de puntos de referencia** implementa:

1. Ecualización adaptativa de histograma:

- **CLAHE** (Contrast Limited Adaptive Histogram Equalization): `clip_limit=2.0`, `tile_size=4`
- Mejora contraste local para resaltar bordes del contorno pulmonar

2. Redimensionamiento: 224×224 píxeles (tamaño de entrada ResNet)

3. Normalización: Media y desviación estándar de ImageNet

Implementación: `src_v2/data/transforms.py`

```
1 # Configuración de ecualización adaptativa
2 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))
```

Nota: Para clasificación se utiliza SAHS (Statistical Asymmetrical Histogram Stretching) aplicado a imágenes warped.

3.1.3 Aumentación de Datos

Durante entrenamiento se aplican:

- Rotación: ± 10
- Traslación: $\pm 5\%$
- Escalado: 0.95–1.05
- Flip horizontal (con corrección de pares simétricos)

3.2 Arquitectura del Modelo

3.2.1 Backbone: ResNet-18

Se utiliza ResNet-18 pre-entrenado en ImageNet como extractor de características:

Componente	Especificación
Backbone	ResNet-18 (pretrained ImageNet)
Coordinate Attention	Sí (después de layer4)
Cabeza de regresión	Deep Head (768 dim)
Dropout	0.3
Salida	30 valores (15 puntos de referencia $\times$ 2 coordenadas)

Cuadro 3: Especificaciones de la arquitectura del modelo.

Implementación: `src_v2/models/resnet_landmark.py`

```

1 class ResNet18Landmarks(nn.Module):
2     def __init__(self, pretrained=True, num_landmarks=15,
3                   use_coord_attention=True, use_deep_head=True,
4                   hidden_dim=768, dropout=0.3):
5         # Backbone ResNet-18
6         self.backbone = models.resnet18(pretrained=pretrained)
7
8         # Coordinate Attention (opcional)
9         if use_coord_attention:
10             self.coord_attention = CoordAttention(512, 512)
11
12         # Cabeza de regresion profunda
13         if use_deep_head:
14             self.regression_head = nn.Sequential(
15                 nn.Linear(512, hidden_dim),
16                 nn.ReLU(),
17                 nn.Dropout(dropout),
18                 nn.Linear(hidden_dim, hidden_dim // 2),
19                 nn.ReLU(),
20                 nn.Dropout(dropout),
21                 nn.Linear(hidden_dim // 2, num_landmarks * 2)
22             )

```

3.2.2 Coordinate Attention

Mecanismo de atención que captura dependencias espaciales de largo alcance:

- Codifica información posicional en los canales
- Captura relaciones horizontales y verticales independientemente
- Mejora la localización precisa de puntos de referencia

3.3 Funciones de Pérdida

Se implementó un sistema de pérdidas combinadas que incorpora restricciones geométricas.

Implementación: src_v2/models/losses.py

3.3.1 Wing Loss (Pérdida Principal)

Función robusta para regresión de puntos de referencia que da mayor peso a errores pequeños:

$$L(x) = \begin{cases} \omega \cdot \ln(1 + |x|/\epsilon) & \text{si } |x| < \omega \\ |x| - C & \text{si } |x| \geq \omega \end{cases} \quad (1)$$

donde $C = \omega - \omega \cdot \ln(1 + \omega/\epsilon)$.

```

1 def wing_loss(pred, target, omega=10, epsilon=2):
2     """
3     Wing Loss: robusto a outliers, sensible a errores pequenos
4     """
5     diff = torch.abs(pred - target)
6     small = diff < omega
7     loss = torch.where(
8         small,
9         omega * torch.log(1 + diff / epsilon),
10        diff - (omega - omega * np.log(1 + omega / epsilon))
11    )
12    return loss.mean()

```

3.3.2 Central Alignment Loss

Penaliza desviaciones de puntos de referencia centrales (L9, L10, L11) respecto al eje vertical:

```

1 def central_alignment_loss(pred, central_indices=[8, 9, 10]):
2     """Penaliza desviacion del eje central (x deberia ser ~0.5)"""
3     central_x = pred[:, central_indices, 0]
4     target_x = 0.5 # Centro normalizado
5     return F.mse_loss(central_x, torch.full_like(central_x,
6         target_x))

```

3.3.3 Soft Symmetry Loss

Penaliza asimetrías excesivas entre pares de puntos de referencia simétricos:

```

1 SYMMETRIC_PAIRS = [(2, 3), (4, 5), (6, 7), (11, 12), (13, 14)]
2
3 def soft_symmetry_loss(pred, margin=6.0):
4     """
5     Penaliza solo cuando asimetria excede margen (6px)
6     Permite asimetria natural en anatomia real
7     """
8     total_loss = 0
9     for left_idx, right_idx in SYMMETRIC_PAIRS:
10        left = pred[:, left_idx]
11        right = pred[:, right_idx]
12        # Distancia al eje central
13        left_dist = torch.abs(left[:, 0] - 0.5)
14        right_dist = torch.abs(right[:, 0] - 0.5)
15        diff = torch.abs(left_dist - right_dist) * 224 # A pixeles
16        # Solo penalizar si excede margen
17        excess = F.relu(diff - margin)
18        total_loss += excess.mean()
19    return total_loss / len(SYMMETRIC_PAIRS)

```


3.3.4 Combined Loss

Combinación ponderada de todas las pérdidas:

```

1 def combined_loss(pred, target,
2                   wing_weight=1.0,
3                   symmetry_weight=0.1,
4                   alignment_weight=0.05):
5     loss = wing_weight * wing_loss(pred, target)
6     loss += symmetry_weight * soft_symmetry_loss(pred)
7     loss += alignment_weight * central_alignment_loss(pred)
8     return loss

```

3.4 Plan de Entrenamiento

Se implementó un esquema de entrenamiento en dos fases con tasas de aprendizaje diferenciadas.

Implementación: src_v2/training/trainer.py

3.4.1 Fase 1: Cabeza Congelada

Parámetro	Valor
Épocas	15
Learning rate	1×10^{-3}
Backbone	Congelado
Objetivo	Estabilizar predicción inicial

```

1 # Fase 1: Solo entrenar cabeza de regresion
2 for param in model.backbone.parameters():
3     param.requires_grad = False
4
5 optimizer = optim.AdamW(model.regression_head.parameters(), lr=1e
    -3)

```

3.4.2 Fase 2: Fine-tuning Diferenciado

Parámetro	Valor
Épocas	100 (con early stopping)
LR backbone	2×10^{-5}
LR cabeza	2×10^{-4}
Scheduler	ReduceLROnPlateau
Early stopping	20 épocas sin mejora

```

1 # Fase 2: Fine-tuning con LR diferenciado
2 for param in model.backbone.parameters():
3     param.requires_grad = True
4
5 optimizer = optim.AdamW([
6     {'params': model.backbone.parameters(), 'lr': 2e-5},
7     {'params': model.regression_head.parameters(), 'lr': 2e-4}
8 ])
9
10 scheduler = optim.lr_scheduler.ReduceLROnPlateau(
11     optimizer, mode='min', factor=0.5, patience=10
12 )

```

3.5 Estrategia de Ensemble

El sistema final combina 4 modelos entrenados con diferentes semillas:

Modelo	Semilla	Error Individual
1	seed123	4.15 px
2	seed321	4.08 px
3	seed111	4.12 px
4	seed666	4.10 px
Ensemble	–	3.61 px

Cuadro 4: Modelos del ensemble y sus errores individuales.

3.5.1 Test-Time Augmentation (TTA)

Durante inferencia, cada imagen se procesa normal y con flip horizontal:

```

1 def predict_with_tta(model, image):
2     # Prediccion original
3     pred_orig = model(image)
4
5     # Prediccion con flip horizontal
6     image_flip = torch.flip(image, dims=[3])
7     pred_flip = model(image_flip)
8
9     # Corregir coordenadas del flip
10    pred_flip[:, :, 0] = 1.0 - pred_flip[:, :, 0]
11
12    # Intercambiar pares simetricos
13    for left, right in SYMMETRIC_PAIRS:
14        pred_flip[:, [left, right]] = pred_flip[:, [right, left]]
15
16    # Promediar
17    return (pred_orig + pred_flip) / 2

```

3.6 Normalización Geométrica (Warping)

Una vez detectados los puntos de referencia, se normaliza la geometría de las imágenes.

3.6.1 Generalized Procrustes Analysis (GPA)

Se calcula la forma estándar a partir de las anotaciones de entrenamiento:

```

1 def gpa_iterative(shapes, max_iter=100, tol=1e-6):
2     """
3     Alinea iterativamente un conjunto de formas
4     hasta converger a la forma media (canonica)
5     """
6     # Inicializar con primera forma
7     mean_shape = shapes[0].copy()
8
9     for _ in range(max_iter):
10        # Alinear todas las formas a la media actual
11        aligned = []
12        for shape in shapes:
13            aligned.append(procrustes_align(shape, mean_shape))
14
15        # Calcular nueva media
16        new_mean = np.mean(aligned, axis=0)
17
18        # Verificar convergencia
19        if np.linalg.norm(new_mean - mean_shape) < tol:
20            break
21        mean_shape = new_mean
22
23    return mean_shape

```

Implementación: src_v2/processing/gpa.py

3.6.2 Triangulación de Delaunay

Se genera una malla de triángulos sobre los puntos de referencia:

```

1 def compute_delaunay_triangulation(landmarks):
2     """
3     Calcula triangulacion de Delaunay sobre los 15 puntos de
4     referencia
5     mas 4 esquinas de la imagen
6     """
7     # Agregar esquinas
8     corners = np.array([[0, 0], [1, 0], [0, 1], [1, 1]])
9     all_points = np.vstack([landmarks, corners])
10
11    # Triangulacion
12    tri = Delaunay(all_points)
13    return tri.simplices

```

3.6.3 Warping Afín por Partes

Cada triángulo se transforma independientemente:

```

1 def piecewise_affine_warp(image, src_landmarks, dst_landmarks,
2   triangles):
3     """
4     Warping afín por partes usando triangulación
5     """
6     output = np.zeros_like(image)
7
8     for tri_indices in triangles:
9         # Obtener vertices del triangulo
10        src_tri = src_landmarks[tri_indices]
11        dst_tri = dst_landmarks[tri_indices]
12
13        # Calcular transformacion afín
14        M = cv2.getAffineTransform(
15            src_tri.astype(np.float32),
16            dst_tri.astype(np.float32)
17        )
18
19        # Aplicar transformacion
20        warped = cv2.warpAffine(image, M, image.shape[:2])
21
22        # Crear mascara del triangulo
23        mask = np.zeros(image.shape[:2], dtype=np.uint8)
24        cv2.fillConvexPoly(mask, dst_tri.astype(np.int32), 255)
25
26        # Combinar
27        output = np.where(mask[..., None], warped, output)
28
29    return output

```

Implementación: src_v2/processing/warp.py

3.6.4 Preprocesamiento SAHS para Clasificación

Las imágenes warped se procesan con SAHS antes de entrenar el clasificador:

```

1 def enhance_contrast_sahs_masked(image, threshold=10):
2     """
3     SAHS (Statistical Asymmetrical Histogram Stretching)
4     Solo aplica a region pulmonar (pixeles > threshold)
5     """
6     # Mascara de region pulmonar
7     mask = image > threshold
8     lung_pixels = image[mask].astype(np.float64)
9
10    gray_mean = np.mean(lung_pixels)
11
12    # Limites asimetricos: factor 2.5 (superior), 2.0 (inferior)
13    above_mean = lung_pixels[lung_pixels > gray_mean]

```

```

14     below_mean = lung_pixels[lung_pixels <= gray_mean]
15
16     std_above = np.sqrt(np.mean((above_mean - gray_mean) ** 2))
17     std_below = np.sqrt(np.mean((below_mean - gray_mean) ** 2))
18
19     max_value = gray_mean + 2.5 * std_above
20     min_value = gray_mean - 2.0 * std_below
21
22     # Estirar histograma solo en region pulmonar
23     enhanced = np.zeros_like(image)
24     transformed = (255 / (max_value - min_value)) * \
25                 (image.astype(np.float64) - min_value)
26     enhanced[mask] = np.clip(transformed[mask], 0, 255)
27
28     return enhanced.astype(np.uint8)

```

Implementación: scripts/generate_warped_sahs_dataset.py

Diferencia con CLAHE: CLAHE se usa en detección de puntos de referencia (mejora bordes locales). SAHS se usa en clasificación (normaliza histograma global preservando patrones patológicos).

4. Resultados y Análisis

4.1 Métricas de Detección de Puntos de Referencia

4.1.1 Error Global

Configuración	Error Medio	Std	Mediana
Mejor modelo individual (seed456)	4.04 px	–	–
Ensemble 4 modelos	3.71 px	2.42 px	3.17 px
Ensemble + TTA (mejor)	3.61 px	2.48 px	3.07 px

Cuadro 5: Comparación de configuraciones de detección de puntos de referencia.

4.1.2 Error por Categoría

Categoría	Error (px)
Normal	3.22
COVID	3.93
Viral_Pneumonia	4.11

Cuadro 6: Error de puntos de referencia por categoría diagnóstica.

4.1.3 Error por Punto de Referencia

Punto de Referencia	Error (px)	Ubicación
L10	2.44	Centro (mejor)
L9	2.76	Centro
L5	2.88	Lateral izq.
L12	5.43	Apex izq. (peor)
L13	5.35	Apex der.
L14	4.39	Base izq.

Cuadro 7: Error de detección por punto de referencia individual.

Observación: Los puntos de referencia centrales (L9, L10, L11) tienen menor error debido a la menor variabilidad anatómica. Los puntos de referencia en ápices (L12, L13) presentan mayor error por la dificultad de definir límites precisos.

4.2 Clasificación

4.2.1 Validación Cruzada (5-fold)

Métrica	Valor	Std
Accuracy	98.60 %	± 0.26 %
F1-macro	98.00 %	± 0.36 %
F1-weighted	98.60 %	± 0.25 %

Cuadro 8: Resultados de validación cruzada 5-fold.

Configuración:

- Dataset: `outputs/warped_lung_best/session_warping`
- Backbone: ResNet-18
- Folds: 5
- Seed: 42

4.3 Objetivos Cumplidos

Objetivo del Plan	Estado
Recopilar y normalizar dataset	✓
Arquitectura ResNet-18 con pérdidas geométricas	✓
Plan de entrenamiento en fases	✓
Protocolo de evaluación	✓
Documentación	✓

Cuadro 9: Verificación de cumplimiento de objetivos.

4.3.1 Extensiones Realizadas (Más allá del Plan)

1. **Ensemble de 4 modelos:** No contemplado originalmente, redujo error de 4.04 a 3.61 px
2. **Test-Time Augmentation:** Mejora adicional con flip horizontal
3. **Sistema de configuración JSON:** Reproducibilidad mejorada
4. **Validación cruzada 5-fold:** Evaluación más robusta
5. **Sistema completo de clasificación:** Incluyendo warping y clasificador

5. Entregables Producidos

5.1 Código Fuente

```
src_v2/
|-- models/
|   |-- resnet_landmark.py      # Arquitectura ResNet-18 +
|       CoordAttention
|   |-- losses.py              # Wing Loss, Symmetry, Alignment
|   +-- classifier.py          # Clasificador COVID-19
|-- training/
|   |-- trainer.py             # Entrenamiento 2 fases
|   +-- callbacks.py           # Early stopping, checkpointing
|-- data/
|   |-- dataset.py             # LandmarkDataset
|   +-- transforms.py          # CLAHE, TTA, aumentacion
|-- processing/
|   |-- gpa.py                 # Generalized Procrustes Analysis
|   +-- warp.py                # Warping afin por partes
|-- evaluation/
|   |-- metrics.py             # Error de puntos de referencia
|   +-- ensemble.py            # Evaluacion de ensemble
+-- cli.py                     # Interfaz de linea de comandos
```

5.2 Modelos Entrenados

4 modelos del ensemble disponibles en `checkpoints/`:

Archivo	Descripción
<code>session10/ensemble/seed123/final_model.pt</code>	Modelo ensemble 1
<code>session13/seed321/final_model.pt</code>	Modelo ensemble 2
<code>repro_split111/session14/seed111/final_model.pt</code>	Modelo ensemble 3
<code>repro_split666/session16/seed666/final_model.pt</code>	Modelo ensemble 4

Cuadro 10: Checkpoints de los modelos del ensemble.

5.3 Configuraciones Reproducibles

```
configs/
|-- ensemble_best.json          # Configuración del mejor ensemble
|-- landmarks_train_base.json   # Hiperparámetros de entrenamiento
|-- warping_best.json           # Parámetros de warping óptimos
+-- classifier_warped_base.json # Configuración del clasificador
```

6. Cronograma Ejecutado

6.1 Mapeo Actividades vs Plan Original

Semana	Plan Original	Actividades Ejecutadas
1 (9-15 Oct)	Revisión estado del arte, análisis dataset	✓ Análisis de 15,153 imágenes, definición de 15 puntos de referencia, configuración de entorno
2 (16-22 Oct)	Arquitectura base, Fase 1	✓ Implementación ResNet-18 + CoordAttention, entrenamiento Fase 1
3 (23-29 Oct)	Fine-tuning, Fases 2-3	✓ Fine-tuning diferenciado, pérdidas geométricas, sweep de semillas
4 (30 Oct-9 Nov)	Experimentos finales, documentación	✓ Ensemble 4 modelos, TTA, sistema de clasificación, documentación

Cuadro 11: Comparación entre cronograma planificado y ejecutado.

6.2 Sesiones de Desarrollo

El desarrollo se documentó en sesiones incrementales:

- **Sesiones 1-10:** Arquitectura base y entrenamiento inicial
- **Sesiones 11-15:** Optimización de hiperparámetros y ensemble
- **Sesiones 16-25:** Proceso de warping y clasificación
- **Sesiones 26-55:** Validación, robustez y documentación

7. Conclusiones

7.1 Objetivos Cumplidos

1. **Detección de puntos de referencia:** Error de 3.61 px con ensemble, superando expectativas
2. **Arquitectura robusta:** ResNet-18 + Coordinate Attention + Deep Head
3. **Pérdidas geométricas:** Wing Loss + Soft Symmetry + Central Alignment
4. **Entrenamiento en fases:** Backbone congelado → Fine-tuning diferenciado
5. **Clasificación COVID-19:** 98.60 % accuracy con validación cruzada

7.2 Contribuciones Técnicas

- Sistema end-to-end desde radiografía hasta clasificación
- Normalización geométrica que reduce variabilidad anatómica
- Configuraciones JSON para reproducibilidad total
- Documentación exhaustiva del proceso experimental

7.3 Limitaciones Identificadas

1. **Domain shift:** Modelos no generalizan a datos externos sin fine-tuning
2. **Puntos de referencia extremos:** Mayor error en ápices pulmonares (L12, L13)
3. **Dependencia de anotaciones:** Calidad limitada por anotaciones manuales

7.4 Trabajo Futuro

1. Explorar arquitecturas más recientes (ViT, ConvNeXt)
2. Domain adaptation para generalización a otros hospitales
3. Detección de puntos de referencia en 3D (CT)
4. Incorporación de incertidumbre en predicciones

Rafael Alejandro Cruz Ovando
Estudiante de Maestría
Benemérita Universidad Autónoma de Puebla

Dr. Leopoldo Altamirano Robles
Director del Proyecto
Instituto Nacional de Astrofísica,
Óptica y Electrónica

Fecha: 28 de enero de 2026